

# Homework of Chapter 5

Name: Kaifeng Tang ID: 524030910124

2026 年 5 月 24 日

## 简答题

1.

距离向量算法核心：Bellman-Ford 方程

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

其中  $D_x(y)$  为从  $x$  到  $y$  的最小路径代价。

对于节点  $z$ ，初始化  $D_z(z) = 0, D_z(v) = 6, D_z(x) = 2$ 。先初始化距离表：

| 起点/终点 | u        | v        | x        | y        | z        |
|-------|----------|----------|----------|----------|----------|
| u     | 0        | 1        | $\infty$ | 2        | $\infty$ |
| v     | 1        | 0        | 3        | $\infty$ | 6        |
| x     | $\infty$ | 3        | 0        | 3        | 2        |
| y     | 2        | $\infty$ | 3        | 0        | $\infty$ |
| z     | $\infty$ | 6        | 2        | $\infty$ | 0        |

由 BF 方程开始更新：

$$D_z(u) = \min\{c(z, v) + D_v(u), c(z, x) + D_x(u)\} = 7$$

同理，

$$D_z(v) = 5, D_z(x) = 2, D_z(y) = 5, D_z(z) = 0$$

不断更新直到所有节点距离向量稳定，最终得到节点  $z$  的距离表：

| z/终点 | u | v | x | y | z |
|------|---|---|---|---|---|
| z    | 6 | 5 | 2 | 5 | 0 |

## 2.

由 Dijkstra 算法, 初始化集合  $N' = t$ ; 对所有  $t$  的邻居,  $D(v) = c(t, v)$ ; 其余令  $D(v) = \infty$ . 然后进行如下循环:

- 在集合  $N'$  以外找到  $D(w)$  的最小值对应的节点  $w$
- 将  $w$  加入集合  $N'$
- 对于  $w$  的不在  $N'$  里的邻居进行更新:  $D(v) = \min\{D(v), D(w) + c(w, v)\}$

直到所有节点都加入  $N'$ , 即可得到  $t$  到其他节点的最短路径。用填表法一步步更新, 最终得到下面的表。

| Step | $N'$    | $D(u), p(u)$ | $D(v), p(v)$ | $D(w), p(w)$ | $D(x), p(x)$ | $D(y), p(y)$ | $D(z), p(z)$ |
|------|---------|--------------|--------------|--------------|--------------|--------------|--------------|
| 0    | t       | 2,t          | 4,t          | $\infty$     | $\infty$     | 7,t          | $\infty$     |
| 1    | tu      |              | 4,t          | 5,u          | $\infty$     | 7,t          | $\infty$     |
| 2    | tuv     |              |              | 5,u          | 7,v          | 7,t          | $\infty$     |
| 3    | tuvw    |              |              |              | 7,v          | 7,t          | $\infty$     |
| 4    | tuvwxy  |              |              |              | 7,v          |              | 19,y         |
| 5    | tuvwxyx |              |              |              |              |              | 15,y         |
| 6    | tuvwxyz |              |              |              |              |              |              |

$tuvwxyz$  即为最小代价生成树, 表征  $t$  到各节点的最短路径; 同时可以得到节点  $t$  到各节点的最短路径长度:

| 节点       | u | v | w | x | y | z  |
|----------|---|---|---|---|---|----|
| $Dist_t$ | 2 | 4 | 5 | 7 | 7 | 15 |

## 3.

1) BGP 路由选择会基于策略控制。一般商业策略中, AS 会更倾向于使用成本更低或收益更高的路径。由于经过 provider 发送流量通常需要付费, 而 peer 之间交换流量成本较低, 因此当 AS100 同时从 AS200 和 AS300 收到到达同一目的网络的路由通告时, 通常会优先选择 AS300 这条 peer 路径。

2) transit-AS 意味着 AS100 不仅转发自己产生或发往自己的流量, 还允许其他 AS 的流量经过 AS100 转发到第三方 AS。则需要调整策略:

- AS100 不仅通告自己的路由, 还可能把从 AS300 学到的路由通告给 AS200, 把从 AS200 学到的路由通告给 AS300。

- 允许将 peer 学到的路由与 provider 路由相互通告。
- 调整本地偏好和路由过滤策略。

可能存在的风险：带宽压力增加，导致链路拥塞；运营成本增加；可能会有安全风险如 DDoS、扫描、攻击流量等安全问题；路由策略复杂度变高。

## Lab5 基于 RIP 的路由配置实验

### 1. 实验目的

本实验基于 Ubuntu 虚拟机中的 Mininet 与 FRR 软件完成 RIP 动态路由配置。通过构建包含 R1、R2、R3、R4、R5 五台路由器的网络拓扑，观察 RIP 协议在正常状态、链路故障状态以及链路恢复状态下的路由表变化，理解距离向量路由协议的基本工作原理、路由收敛过程以及链路故障后的动态路由更新机制。

### 2. 实验过程及分析

#### (1) 搭建实验拓扑

本实验使用 5 台路由器节点。由实验要求的链路关系图，可以设置各链路的对应网段，如下表所示。每台路由器通过 FRR 的 zebra 和 ripd 进程运行 RIP 协议，并在各自的配置文件中声明自身的直连网段，使路由器之间能够交换路由信息。

| 链路    | 对应网段        | 接口 ip                      |
|-------|-------------|----------------------------|
| R1-R2 | 10.0.1.0/24 | R1: 10.0.1.1, R2: 10.0.1.2 |
| R2-R3 | 10.0.2.0/24 | R2: 10.0.2.1, R3: 10.0.2.2 |
| R4-R5 | 10.0.3.0/24 | R4: 10.0.3.1, R5: 10.0.3.2 |
| R1-R4 | 10.0.4.0/24 | R1: 10.0.4.1, R4: 10.0.4.2 |
| R2-R5 | 10.0.5.0/24 | R2: 10.0.5.1, R5: 10.0.5.2 |
| R3-R5 | 10.0.6.0/24 | R3: 10.0.6.1, R5: 10.0.6.2 |

由示例代码编写实验拓扑运行脚本 `rip_lab.py`。

#### (2) 启动拓扑

运行拓扑脚本，进入 mininet 环境，并等待 RIP 协议完成初始收敛。随后分别查看 R1、R2、R3、R4、R5 的 ip route 路由表，如图 1 所示。

从初始化路由表截图可以看到，每台路由器的路由表中均包含两类路由：

- `proto kernel`, 表示该路由器的直连路由，由系统根据接口 IP 自动生成。
- `proto rip`, 表示该路由器通过 RIP 协议动态学习到的非直连网段路由。

例如在 R1 的路由表中，直连路由为：

```
10.0.1.0/24 dev r1-eth0 proto kernel
10.0.4.0/24 dev r1-eth1 proto kernel
```

同时，R1 还通过 RIP 协议学习到其他的 RIP 非直连路由：

```
10.0.2.0/24 via 10.0.1.2 dev r1-eth0 proto rip
10.0.3.0/24 via 10.0.4.2 dev r1-eth1 proto rip
10.0.5.0/24 via 10.0.1.2 dev r1-eth0 proto rip
10.0.6.0/24 via 10.0.1.2 dev r1-eth0 proto rip
```

这说明 R1 已经通过 RIP 协议学习到了 R2-R3、R4-R5、R2-R5、R3-R5 等非直连网段的路由信息。对 R2 R5 的分析同理。由此可以判断，初始状态下 RIP 已经完成路由收敛。

```
mininet> r1 ip route
10.0.1.0/24 dev r1-eth0 proto kernel scope link src 10.0.1.1
10.0.2.0/24 nhid 10 via 10.0.1.2 dev r1-eth0 proto rip metric 20
10.0.3.0/24 nhid 13 via 10.0.4.2 dev r1-eth1 proto rip metric 20
10.0.4.0/24 dev r1-eth1 proto kernel scope link src 10.0.4.1
10.0.5.0/24 nhid 10 via 10.0.1.2 dev r1-eth0 proto rip metric 20
10.0.6.0/24 nhid 10 via 10.0.1.2 dev r1-eth0 proto rip metric 20
mininet> r2 ip route
10.0.1.0/24 dev r2-eth0 proto kernel scope link src 10.0.1.2
10.0.2.0/24 dev r2-eth1 proto kernel scope link src 10.0.2.1
10.0.3.0/24 nhid 19 via 10.0.5.2 dev r2-eth2 proto rip metric 20
10.0.4.0/24 nhid 14 via 10.0.1.1 dev r2-eth0 proto rip metric 20
10.0.5.0/24 dev r2-eth2 proto kernel scope link src 10.0.5.1
10.0.6.0/24 nhid 16 via 10.0.2.2 dev r2-eth1 proto rip metric 20
mininet> r3 ip route
10.0.1.0/24 nhid 10 via 10.0.2.1 dev r3-eth0 proto rip metric 20
10.0.2.0/24 dev r3-eth0 proto kernel scope link src 10.0.2.2
10.0.3.0/24 nhid 13 via 10.0.6.2 dev r3-eth1 proto rip metric 20
10.0.4.0/24 nhid 10 via 10.0.2.1 dev r3-eth0 proto rip metric 20
10.0.5.0/24 nhid 10 via 10.0.2.1 dev r3-eth0 proto rip metric 20
10.0.6.0/24 dev r3-eth1 proto kernel scope link src 10.0.6.1
mininet> r4 ip route
10.0.1.0/24 nhid 10 via 10.0.4.1 dev r4-eth0 proto rip metric 20
10.0.2.0/24 nhid 10 via 10.0.4.1 dev r4-eth0 proto rip metric 20
10.0.3.0/24 dev r4-eth1 proto kernel scope link src 10.0.3.1
10.0.4.0/24 dev r4-eth0 proto kernel scope link src 10.0.4.2
10.0.5.0/24 nhid 12 via 10.0.3.2 dev r4-eth1 proto rip metric 20
10.0.6.0/24 nhid 12 via 10.0.3.2 dev r4-eth1 proto rip metric 20
mininet> r5 ip route
10.0.1.0/24 nhid 16 via 10.0.5.1 dev r5-eth1 proto rip metric 20
10.0.2.0/24 nhid 17 via 10.0.6.1 dev r5-eth2 proto rip metric 20
10.0.3.0/24 dev r5-eth0 proto kernel scope link src 10.0.3.2
10.0.4.0/24 nhid 18 via 10.0.3.1 dev r5-eth0 proto rip metric 20
10.0.5.0/24 dev r5-eth1 proto kernel scope link src 10.0.5.2
10.0.6.0/24 dev r5-eth2 proto kernel scope link src 10.0.6.2
```

图 1: R1-R5 初始化路由表

### (3) 测试连通性

从 R1 分别 ping R3 和 R5。用 R3 在 R3-R5 链路上的接口地址 10.0.6.1 以及 R5 在 R4-R5 链路上的接口地址 10.0.3.2 测试。从连通性测试截图图 2 中可以看到，两次 ping 测试均显示

```
3 packets transmitted, 3 received, 0% packet loss
```

这说明 R1 到 R3、R1 到 R5 均能够正常通信。R1 虽然并不直接连接 R3 和 R5 的目标接口，但由于 RIP 已经学习到了对应的路由，数据包可以通过动态路由转发到目标网络。这进一步说明了 RIP 路由配置有效。

```
mininet> r1 ping -c 3 10.0.6.1
PING 10.0.6.1 (10.0.6.1) 56(84) bytes of data.
64 bytes from 10.0.6.1: icmp_seq=1 ttl=63 time=4.42 ms
64 bytes from 10.0.6.1: icmp_seq=2 ttl=63 time=4.19 ms
64 bytes from 10.0.6.1: icmp_seq=3 ttl=63 time=4.24 ms

--- 10.0.6.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 4.188/4.282/4.418/0.098 ms
mininet> r1 ping -c 3 10.0.3.2
PING 10.0.3.2 (10.0.3.2) 56(84) bytes of data.
64 bytes from 10.0.3.2: icmp_seq=1 ttl=63 time=4.15 ms
64 bytes from 10.0.3.2: icmp_seq=2 ttl=63 time=4.07 ms
64 bytes from 10.0.3.2: icmp_seq=3 ttl=63 time=4.19 ms

--- 10.0.3.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 4.067/4.135/4.194/0.052 ms
```

图 2: 初始化连通性测试截图

### (4) 断开 R2-R5 链路后的路由更新

用 `link r2 r5 down` 模拟链路故障，断开 R2 与 R5 之间的链路，即网段 10.0.5.0/24 对应的链路失效。断链后再次查看 R1、R2、R5 的路由表。从截图图 3 可以观察到明显变化：

- R2 与 R5 的直连路由发生变化：断链前，R2 与 R5 的路由表中存在 R2-R5 之间的直连链路 `10.0.5.0/24 dev r2-eth2 proto kernel`，断链后，路由表中已经不再显示 `10.0.5.0/24 dev r2-eth2`，说明 R2-R5 链路已经被成功断开。
- R2 到 R5 相关网络的路由发生改变：R2 已不再通过 R2-R5 直连链路到达 R5 方向，而是改为通过 R3 转发：

```
10.0.3.0/24 via 10.0.2.2 dev r2-eth1 proto rip
10.0.6.0/24 via 10.0.2.2 dev r2-eth1 proto rip
```

即路径变为  $R2 \rightarrow R3 \rightarrow R5$ 。

- 同理，R5 到 R1、R2 相关网段的路径变为通过 R3 转发：路径变为 R5 → R3 → R2 → R1。

因此，虽然 R2-R5 链路发生故障，但由于拓扑中仍然存在其他可达路径，RIP 能够重新选择路由，使网络保持连通。

```
mininet> link r2 r5 down
mininet> r1 ip route
10.0.1.0/24 dev r1-eth0 proto kernel scope link src 10.0.1.1
10.0.2.0/24 nhid 10 via 10.0.1.2 dev r1-eth0 proto rip metric 20
10.0.3.0/24 nhid 13 via 10.0.4.2 dev r1-eth1 proto rip metric 20
10.0.4.0/24 dev r1-eth1 proto kernel scope link src 10.0.4.1
10.0.6.0/24 nhid 10 via 10.0.1.2 dev r1-eth0 proto rip metric 20
mininet> r2 ip route
10.0.1.0/24 dev r2-eth0 proto kernel scope link src 10.0.1.2
10.0.2.0/24 dev r2-eth1 proto kernel scope link src 10.0.2.1
10.0.3.0/24 nhid 16 via 10.0.2.2 dev r2-eth1 proto rip metric 20
10.0.4.0/24 nhid 14 via 10.0.1.1 dev r2-eth0 proto rip metric 20
10.0.6.0/24 nhid 16 via 10.0.2.2 dev r2-eth1 proto rip metric 20
mininet> r5 ip route
10.0.1.0/24 nhid 17 via 10.0.6.1 dev r5-eth2 proto rip metric 20
10.0.2.0/24 nhid 17 via 10.0.6.1 dev r5-eth2 proto rip metric 20
10.0.3.0/24 dev r5-eth0 proto kernel scope link src 10.0.3.2
10.0.4.0/24 nhid 18 via 10.0.3.1 dev r5-eth0 proto rip metric 20
10.0.6.0/24 dev r5-eth2 proto kernel scope link src 10.0.6.2
```

图 3: 断链后更新的部分路由表

#### (5) 断链后连通性测试

再次从 R1 测试到 R3 和 R5 的连通性，得到图 4。从测试结果可以看到，两次 ping 仍然显示可达。这说明 RIP 协议已经根据链路故障重新更新路由表，并选择了新的可用路径。断链后的连通性测试证明 RIP 已完成重新收敛。

```
mininet> r1 ping -c 3 10.0.6.1
PING 10.0.6.1 (10.0.6.1) 56(84) bytes of data.
64 bytes from 10.0.6.1: icmp_seq=1 ttl=63 time=4.15 ms
64 bytes from 10.0.6.1: icmp_seq=2 ttl=63 time=4.08 ms
64 bytes from 10.0.6.1: icmp_seq=3 ttl=63 time=4.15 ms

--- 10.0.6.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 4.080/4.126/4.150/0.032 ms
mininet> r1 ping -c 3 10.0.3.2
PING 10.0.3.2 (10.0.3.2) 56(84) bytes of data.
64 bytes from 10.0.3.2: icmp_seq=1 ttl=63 time=4.10 ms
64 bytes from 10.0.3.2: icmp_seq=2 ttl=63 time=4.09 ms
64 bytes from 10.0.3.2: icmp_seq=3 ttl=63 time=4.18 ms

--- 10.0.3.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 4.087/4.122/4.177/0.039 ms
```

图 4: 断链后连通性测试

### (6) 恢复 R2-R5 链路后的路由更新

用 `link r2 r5 up` 恢复 R2-R5 链路。从恢复后的路由表截图图 5 可以看出，R2 和 R5 的直连网段 10.0.5.0/24 已经重新出现，说明 R2-R5 链路已经恢复成功。同时，一些其他部分的路由又重新通过对 R2-R5 链路的学习选择，例如 R5 到 R1 方向的路由：

```
10.0.1.0/24 via 10.0.5.1 dev r5-eth1 proto rip
```

所表示的路径为  $R5 \rightarrow R2 \rightarrow R1$ ，R2 和 R5 重新拥有 10.0.5.0/24 的直连路由。

## 3. 分析 RIP 协议在链路故障时的收敛时间及路由环路避免机制

### (1) RIP 收敛时间分析

在本次实验中，初始状态下所有链路正常，五台路由器通过 RIP 相互交换路由信息，最终每台路由器都学习到了所有非直连网段的路由。

当 R2-R5 链路断开后，R2 和 R5 失去了 10.0.5.0/24 的直连路由。随后 RIP 根据其邻居通告的信息重新选择路径。例如 R2 可以通过 R3 到达 R5，R5 也可以通过 R3 到达 R2 和 R1。因此，网络并未完全中断，而是经过短暂收敛后恢复正常转发。

当 R2-R5 链路重新连接后，R2 与 R5 再次恢复直连关系，并重新通过 RIP 通告该链路。其他路由器接收到新的路由信息后，再次更新自己的路由表，使路由状态回到接近初始状态。

### (2) 路由环路避免机制

RIP 作为一种距离向量协议，在链路故障时可能出现路由环路或“计数到无穷”问题。为了缓解这些问题，RIP 使用以下机制：

- 最大跳数限制：RIP 规定最大跳数为 15，跳数达到 16 时表示不可达，限制错误路由无限传播。
- 水平分割性质：路由器不会把从某个接口学习到的路由再从同一个接口通告回去，从而减少两个路由器之间形成简单环路的可能。
- 当某条路由失效时，路由器会向邻居通告该路由不可达，加快对错误路由的清除。
- 当链路状态发生变化时，路由器不必完全等待周期性更新，而是可以主动向邻居发送更新信息。

在本实验中，断开 R2-R5 链路后，RIP 能够重新选择 R2-R3-R5 或 R1-R4-R5 等替代路径，说明路由器之间完成了动态更新，没有出现明显的长期路由环路。

## 4. 思考题解答

Q: 如果将所有链路代价改为非对称值（如  $R1-R2=1$ ， $R2-R1=5$ ），RIP 协议能否正常工作？

A: 可能无法正常工作。RIP 本身不适合精确处理非对称链路代价。RIP 的度量标准主要是跳数，而非链路带宽、延迟或方向性代价。如果网络中存在非对称链路代价（如  $R1-R2=1$ ， $R2-R1=5$ ），那么实际网络代价具有方向性。但是 RIP 的默认跳数度量无法准确表达这种方向差异，因此无法选择最低代价路径。

因此，RIP 只能维持基本连通性，不能很好地根据非对称链路代价选择最优路径。